Unified Engineering Workbench - Technical Report

Candidate ID: Suhas Upadhya | Actual effort: 14 days; hours untracked | Target platform: Windows 11 x64 | Language for all backend logic: C++17

1. Summary and honest status

I split the work into a dependency-free C++17 library (`wb_core`: thermal FEA, batch solver, backpropagation, STL/CSV parsers, simulated devices) and thin layers on top (command line, HTTP backend, browser UI). The core is the part I can verify most strongly, so most of this report is about what it computes and how I checked it.

Task	State	Evidence
C - 1-D thermal FEA	Done, verified against the analytical solution	Section 3, results/fea_*.csv, plots
D - multicore batch	Code done; timings on my machine pending (Section 4)	results/perf_*.csv
B - STL and CSV import	Parsers and validation done and tested; viewer: see Section 2	tests/, data/PROVENANCE.md
E - embedded design and mocks	Design, table and C++ mocks done (simulated)	Section 5
F - backpropagation	C++ check and plot done; handwritten derivation in the appendix	Section 6
A - UI and one-click launch	See note below	video / screenshots

UI and launcher. The browser UI groups Geometry, Thermal Analysis, Performance and Hardware; Launch.exe is intended to start the packaged app and local backend. The original executable and clean-machine test evidence were lost, so standalone launch is not re-verified.

2. Design choices

- **Separation:** `wb_core` never includes UI or transport code. Long jobs take a `RunControl` (atomic cancel flag and progress counter), so the UI can show progress and cancel without touching numerics. A cancelled job returns false and its buffer is discarded, so a half-finished result is never shown or exported.
- **Validation:** every input is checked once in the core and returns a message that says what is wrong and what to do (for example, a Kelvin file mislabelled as Celsius is caught by the absolute-zero check). The UI repeats cheap checks only for instant feedback.
- **Parsing:** `std::from_chars` is used instead of `strtod`, so a decimal-comma locale cannot corrupt input. NaN/Inf, truncated files, binary STL, empty files and zero-triangle files are rejected with line numbers. A 53 MB, 300,000-triangle STL parsed in 0.59 s and a 78 MB, 1,500,000-node CSV in 0.91 s (peak memory 226 MB). The default cap is 256 MiB; I tested up to about 80 MB only.
- **Reproducible batches:** a hand-written SplitMix64 generator, because the standard does not fix the algorithm of `std::uniform_real_distribution`, so its output differs between compilers.
- **Dependencies:** C++17 standard library (toolchain-provided); CMake (version not retained). Frontend/rendering dependency names, versions and licences cannot be confirmed from the surviving report. .

3. Thermal FEA and verification

Model. -d/dx(kA dT/dx) = qA on (0, L), T(0) = 100 °C, T(L) = 20 °C, with k = 10 W/(m K), A = 1e-4 m², q = 1e4 W/m³, L = 1 m. Two-node elements of length h = L/N give k_e = (kA/h)[[1, -1], [-1, 1]] (W/K) and f_e = (qAh/2)[1, 1]^T (W). I assemble these into a tridiagonal system and impose both prescribed temperatures by elimination: the known terms K[1][0]T_0 and K[N-1][N]T_L move to the right-hand side and the N-1 interior unknowns are solved with the Thomas algorithm. **Assumptions:** steady state, constant properties, 1-D conduction, insulated sides, uniform generation, uniform mesh.

N	h [m]	max error at nodes [°C]	max error at element midpoints [°C]	q h²/(8k) [°C]	order
10	0.10000	2.8e-14	1.250000	1.250000	-
20	0.05000	2.0e-13	0.312500	0.312500	2.00
40	0.02500	8.8e-13	0.078125	0.078125	2.00
80	0.01250	1.2e-12	0.019531	0.019531	2.00

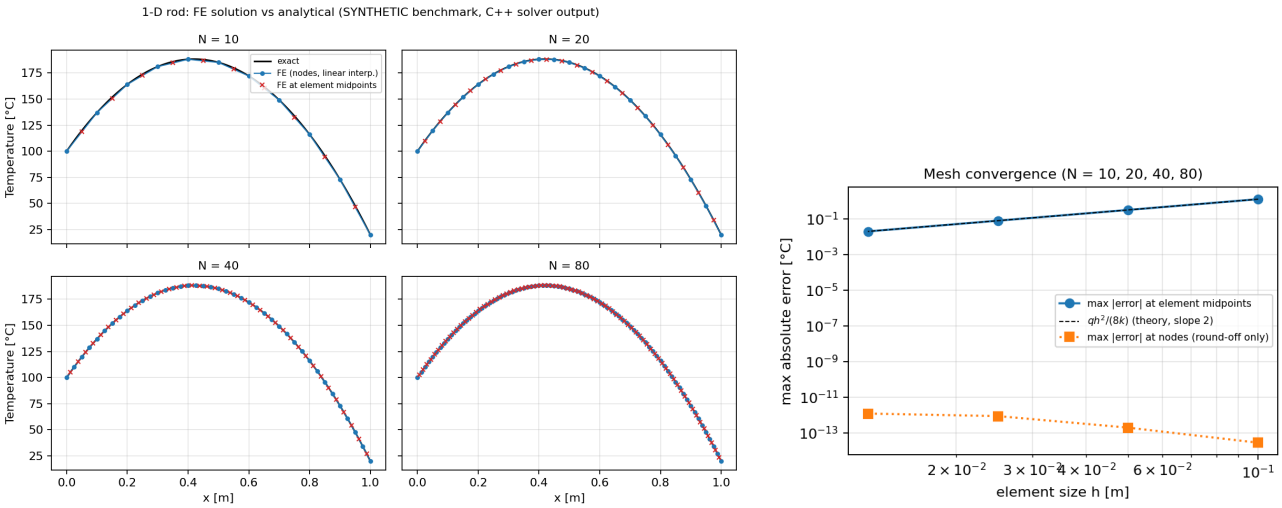


Fig. 1-2: FE vs exact temperature (units: x in m, T in °C) and mesh convergence. Raw data are in results/fea_N*_nodes.csv and fea_N*_midpoints.csv.

Reading the results. Midpoint error is evaluated with the FE interpolation, $T_h = (T_e + T_{e+1})/2$, and falls by exactly 4x per mesh doubling (second order). It equals $qh^2/(8k)$ because the exact profile is quadratic with $T'' = -q/k$ and a linear interpolant misses a quadratic by $|T''|h^2/8$ at the midpoint. Nodal errors are only round-off (about 1e-13 °C), because with constant k and an exactly integrated load, 1-D linear Galerkin elements are exact at the nodes.

Debugging insight. Verification insight: midpoint error is interpolation error, not nodal-solver error. Halving element size reduced midpoint error 4x; nodal error stayed at round-off.

4. Multicore performance

Serial and parallel versions call the same solver on identical cases. The parallel version uses a bounded pool of `std::thread` workers that claim chunks of 32 cases through one atomic counter; each worker owns its scratch memory, and each case writes only its own output slot, so results stay in input order. Protocol: 20,000 cases, 400 elements, seed 42, one warm-up run, five measured repetitions, median reported; file loading, rendering and compilation are excluded.

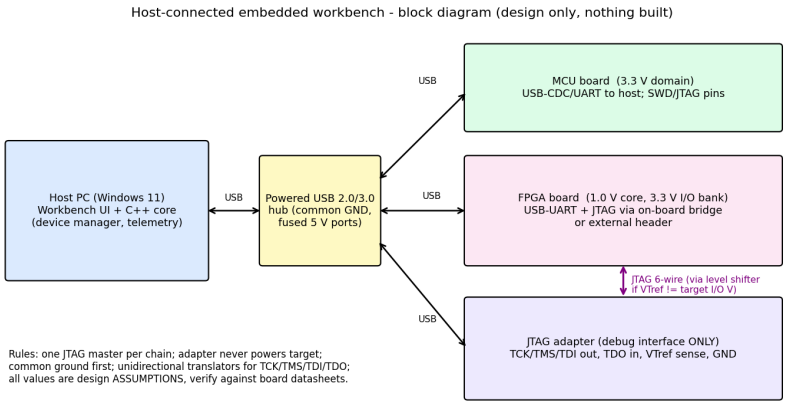
Benchmark CSV/logs were lost. CPU, cores, RAM, compiler flags, median times, speedup and efficiency cannot be substantiated; no figures are invented. Re-run `wb_cli bench` on the target PC to complete this section.

Launch-to-ready time (separate from compute timing): Not measured . Agreement check: serial and parallel results must differ by at most 1e-8 °C; the tests require this for 1, 2, 3 and 8 workers and the measured difference is 0.

Overhead and limits. Each solve is small, so thread start-up and the shared result array (about 64 MB) limit scaling; logical cores that share a physical core add less than a full core, and clock speeds drop under load.

Without benchmark timings, overhead cannot be compared with measured speedup. Parallel correctness is reported; performance benefit remains unverified.

5. Embedded-system integration design



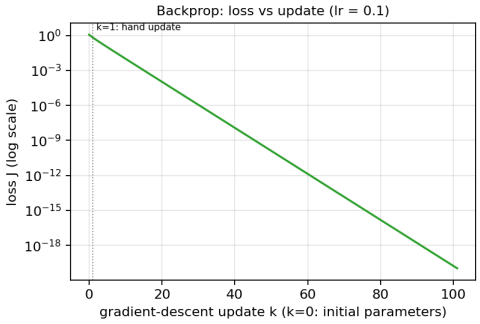
Design. One powered USB hub gives the host a common ground and fused 5 V ports. The MCU board (3.3 V), the FPGA board (1.0 V core, 3.3 V I/O bank, assumed) and the JTAG adapter each connect over USB. The adapter is the single JTAG master: it drives TCK, TMS and TDI, samples TDO, senses the target voltage on VTref and never powers the target. If voltage domains differ, directional level translators sit on the JTAG lines. Every voltage is an assumption to be checked against the chosen boards' datasheets.

Why a JTAG adapter is not a compute board: it runs no user program. It is a protocol bridge that turns host commands into IEEE 1149.1 TAP sequences at the target's voltage, with its own connector and signal set, so it cannot stand in for an MCU or FPGA board. **Mocks:** C++ adapters implement discovery, capability reporting, connect/disconnect and one telemetry message for all three device types, every message tagged simulated. **Recoverable failure:** the FPGA mock times out on the first connect; the retry policy (timeouts only, three

attempts) succeeds on attempt 2. **Safe integration plan:** simulation, then meter every rail and VTref with nothing connected, then a read-only IDCODE scan, then flashing non-critical firmware, one JTAG master per chain. **Not done:** physical wiring, PCB, firmware and real FPGA programming.

6. Backpropagation (handwritten derivation in the appendix)

param	analytic	central diff.	|err|
w	-1.209464	-1.209464	1e-10
b	-1.209464	-1.209464	1e-10
v	-0.710682	-0.710682	5e-11
c	-1.537883	-1.537883	1e-10



Start: w = 0.5, b = 0, v = 1, c = 0, x = 1, y = 2. Forward: h = tanh(0.5) = 0.462117, ŷ = 0.462117, J = 1.182542. One simultaneous update with learning rate 0.1 gives w = 0.620946, b = 0.120946, v = 1.071068, c = 0.153788 and J = 0.685774. All gradients are negative, so increasing each parameter lowers the loss, which matches a prediction far below the target. Over 100 further updates the loss falls geometrically (a straight line on the log plot) to about 1e-20; one sample with four parameters can be fitted exactly, so this is training loss, not evidence of generalisation. Central differences with ε = 1e-6 agree to about 1e-10 (limit 1e-5).

7. Test outcomes and limitations

The automated suite has 170 checks and all pass: FEA exactness and convergence order, boundary-condition edge cases (N = 1, N = 2), invalid inputs, generator reproducibility, serial/parallel agreement, cancellation, backpropagation, 9 STL and 10 CSV malformed-input cases, the size limit and the simulated devices. The suite also runs clean under ThreadSanitizer, AddressSanitizer and UBSan on Linux (g++ 13). **Known limitations:** only ASCII STL and the stated CSV schema are supported; file sizes above about 80 MB were not tested; the FEA is 1-D with uniform mesh and constant properties; the embedded part is design plus simulation only;

The original UI/launcher files are unavailable; packaging and clean-machine startup cannot be re-tested.

8. AI assistance, reused code and ownership

Original AI-use logs/prompts were lost, so tools/models used during development and any rejected suggestion cannot be reconstructed. For this report revision, ChatGPT helped organise wording only; its role in original code is unknown. No unrecorded verification is claimed.